



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
BASES DE DATOS
SEMESTRE 2027 - 1



TAREA 03
Conceptos Orientados a PostgreSQL

ALUMNO	<i>Arzate Rios Adrián Axel</i>
PROFESOR	<i>Ing. Fernando Arreola Franco</i>
GRUPO	<i>01</i>
FECHA DE ENTREGA	<i>31 de Agosto de 2026</i>

1. Conceptos Orientados a PostgreSQL

Investigar:

- Concepto de roles y privilegios.
- Cómo dar y quitar roles.
- Privilegios a nivel sistema y objeto.
- Diferencia entre roles y usuario.

1.1. Roles y Privilegios

- **Roles:** Un rol puede considerarse como un usuario de la base de datos o un grupo de usuarios, según su configuración. Los roles pueden ser propietarios de objetos de la base de datos (por ejemplo, tablas y funciones) y asignar privilegios sobre esos objetos a otros roles para controlar quién tiene acceso a qué objetos. Además, es posible otorgar la pertenencia a un rol a otro, permitiendo así que el rol miembro utilice los privilegios asignados a otro rol. También cabe mencionar que **PostgreSQL** gestiona los permisos de acceso a la base de datos mediante el concepto de roles
- **Privilegios:** Son los permisos otorgados a un usuario o rol para ejecutar instrucciones SQL específicas o acceder a determinados objetos. Existen diferentes tipos de privilegios: SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER, CREATE, CONNECT, TEMPORARY, EXECUTE, USAGE, SET, ALTER SYSTEM, y MAINTAIN. Los privilegios aplicables a un objeto en particular varían según su tipo (tabla, función, etc.).

1.2. Roles, cómo dar y quitar

- **¿Cómo dar roles?** Para otorgar un rol a un usuario o a otro rol en PostgreSQL se utiliza el comando GRANT. Al otorgar un rol, el usuario pasa a ser miembro de dicho rol y puede obtener los privilegios asociados a este.

Sintaxis:

```
GRANT role_name [, ...] TO role_name [, ...]
```

Donde:

role_name representa el nombre del rol que se desea otorgar y, después de TO, el nombre del usuario o rol que recibirá la membresía.

Ejemplo:

```
GRANT administrador TO Arreola;
```

En este ejemplo, se otorga el rol administrador al usuario Arreola.

- **¿Cómo quitar roles?** Para quitar un rol previamente otorgado a un usuario o a otro rol en PostgreSQL se utiliza el comando REVOKE. Esto elimina la membresía del usuario en dicho rol.

Sintaxis:

```
REVOKE role_name [, ...] FROM role_name [, ...]
```

Donde:

role_name representa el nombre del rol que se desea quitar y, después de FROM, el nombre del usuario o rol al que se le retirará la membresía.

Ejemplo:

```
REVOKE administrador FROM Axel;
```

En este ejemplo, se quita el rol administrador al usuario Axel.

Es importante diferenciar REVOKE de DROP ROLE. El comando REVOKE quita la membresía de un usuario en un rol, mientras que DROP ROLE elimina completamente el rol de la base de datos.

Sintaxis de DROP ROLE:

```
DROP ROLE [ IF EXISTS ] name [, ...]
```

1.3. Privilegios a nivel Sistema y Objeto

En PostgreSQL los privilegios determinan qué acciones puede realizar un usuario o rol dentro de la base de datos. Estos permisos pueden relacionarse con capacidades generales del sistema o con el acceso a objetos específicos de la base de datos.

- **Privilegios a nivel sistema:**

En PostgreSQL, algunas capacidades administrativas se manejan mediante atributos de los roles y roles predefinidos. Estos permiten realizar acciones que afectan al sistema o a la administración de la base de datos y no solamente a un objeto específico.

Algunos ejemplos de estas capacidades son:

- SUPERUSER: permite al rol omitir prácticamente todas las comprobaciones de permisos.
- CREATEDB: permite crear nuevas bases de datos.
- CREATEROLE: permite administrar otros roles.
- LOGIN: permite que un rol pueda iniciar sesión en PostgreSQL.

Por ejemplo, para permitir que un usuario pueda crear bases de datos:

```
ALTER ROLE usuario1 CREATEDB;
```

También existen roles predefinidos de PostgreSQL que proporcionan determinadas capacidades privilegiadas, como pg_read_all_data, pg_write_all_data y pg_signal_backend.

- **Privilegios a nivel objeto:**

Los privilegios a nivel objeto permiten controlar las operaciones que un usuario o rol puede realizar sobre un objeto específico de la base de datos, como tablas, secuencias, esquemas, funciones o bases de datos.

Entre los principales privilegios se encuentran:

- SELECT: permite consultar datos.
- INSERT: permite insertar nuevos registros.
- UPDATE: permite modificar registros.
- DELETE: permite eliminar registros.
- TRUNCATE: permite vaciar una tabla.
- REFERENCES: permite crear restricciones de clave foránea que hagan referencia al objeto.
- TRIGGER: permite crear triggers sobre una tabla.
- EXECUTE: permite ejecutar una función o procedimiento.
- USAGE: permite utilizar determinados objetos, como esquemas y secuencias.

Para otorgar estos privilegios se utiliza la sentencia GRANT.

Sintaxis:

```
GRANT privilegio ON objeto TO rol;
```

Ejemplo:

```
GRANT SELECT, INSERT ON empleados TO usuario1;
```

En este ejemplo, usuario1 puede consultar e insertar información en la tabla empleados, pero no obtiene automáticamente permisos para modificar o eliminar sus registros.

Para quitar los privilegios se utiliza REVOKE.

Ejemplo:

```
REVOKE INSERT ON empleados FROM usuario1;
```

Después de ejecutar esta instrucción, usuario1 conserva el privilegio SELECT, pero pierde el permiso INSERT sobre la tabla empleados.

1.4. Diferencia entre roles y usuario

En PostgreSQL, los usuarios y los roles se manejan bajo el mismo concepto de **rol**. Un rol puede actuar como un usuario de la base de datos o como un grupo que contiene a otros roles.

La principal diferencia es que un **usuario** es un rol que tiene el atributo `LOGIN`, lo que le permite iniciar una sesión y conectarse a la base de datos. Por otro lado, un **rol** sin el atributo `LOGIN` normalmente se utiliza para agrupar privilegios y posteriormente otorgarlos a otros roles o usuarios.

Por ejemplo, para crear un usuario se puede utilizar:

```
CREATE ROLE usuario1 LOGIN PASSWORD 'contraseña';
```

Mientras que un rol utilizado para agrupar privilegios se puede crear como:

```
CREATE ROLE administrador;
```

Después, el rol administrador puede ser otorgado al usuario:

```
GRANT administrador TO usuario1;
```

De esta manera, `usuario1` se convierte en miembro del rol `administrador` y puede obtener los privilegios asociados a dicho rol.

PostgreSQL también proporciona el comando `CREATE USER`. Este funciona de manera similar a `CREATE ROLE`, pero incluye `LOGIN` de manera predeterminada. Por esta razón, en PostgreSQL un usuario puede considerarse como un rol con permiso para iniciar sesión.

1.5. ¿Cómo conectarse a una base de datos?

Para conectarse a una base de datos PostgreSQL es necesario contar con los datos de conexión y un software cliente que permita establecer la comunicación con el servidor.

- **Software:** Se necesita un cliente compatible con PostgreSQL. Uno de los principales es `psql`, que permite conectarse y ejecutar instrucciones SQL desde la terminal.
- **IP / Dirección:** Es la dirección del servidor donde se encuentra PostgreSQL. Puede ser una dirección IP, como `192.168.1.10`, o un nombre de host, como `localhost`.
- **Contraseña:** Es la contraseña asociada al usuario con el que se realizará la conexión. Puede ser solicitada por `psql` al momento de conectarse.
- **Nombre de usuario:** Es el nombre del rol de PostgreSQL que cuenta con el atributo `LOGIN` y que se utilizará para establecer la conexión.
- **Puerto:** Es el puerto TCP en el que el servidor PostgreSQL acepta conexiones. El puerto predeterminado es `5432`.
- **Nombre de la base de datos:** Indica la base de datos específica a la que se desea realizar la conexión.

Ejemplo:

```
psql -U nombre_usuario -h IP -p puerto -d nombre_BD -W
```

Donde:

- -U indica el nombre del usuario.
- -h indica la dirección IP o nombre del servidor.
- -p indica el puerto de conexión.
- -d indica el nombre de la base de datos.
- -W hace que psql solicite la contraseña antes de realizar la conexión.

Por ejemplo:

```
psql -U usuario1 -h 192.168.1.10 -p 5432 -d laboratorio -W
```